

C Programming

Structure: Ch.13

Definition & initialization | Structures & pointers | typedef

Classification of Data Types

C DATA TYPE

BASIC DATA TYPE

Built-in fundamental types.

int	Integer (usually 4 bytes)
long	Long integer (4 or 8 bytes)
float	Single-precision floating point (4 bytes)
double	Double-precision floating point (8 bytes)
char	Character (1 byte)
...	...

DERIVED DATA TYPE

Types derived from basic types.



pointer

Stores address of a variable.



struct

Collection of different types under one name.



union

Different members share the same memory.

...

Other derived types

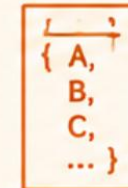
USER-DEFINED DATA TYPE

Defined by the programmer.



typedef

Creates a new name for an existing type.



enum

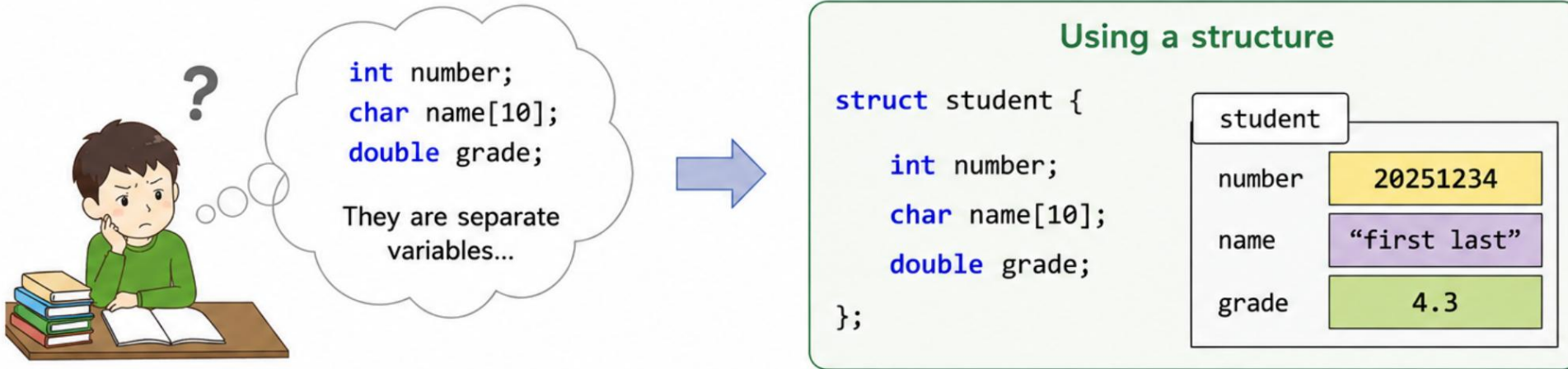
Set of named integer constants.

...

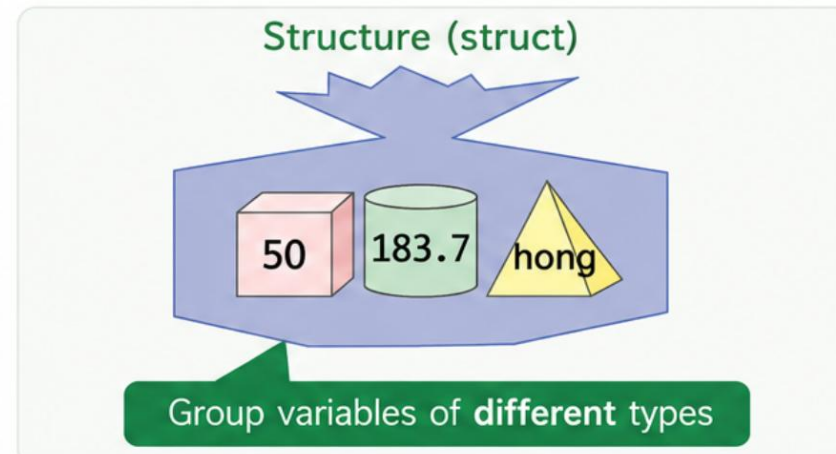
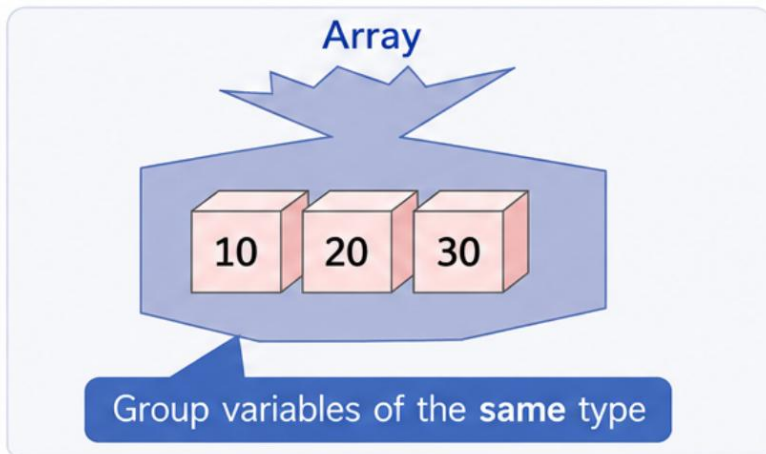
Other user-defined types

The Need for Structures

- How to gather data about a student into one place ?



Arrays and Structures



Structures allow you to **group related variables** together under one name.

Structure Definition & Declaration

1. Structure Definition (creates a blueprint)

- A structure definition describes the **layout** of a new type.

```
struct student {  
    int    number;    // student number  
    char   name[10];  // name  
    double grade;     // grade  
};
```

Keyword

Name of the structure

Members (fields)

Semicolon required



Like a mold for making waffles.
It does **NOT** create any memory.



Example

```
struct student {  
    int number;  
    char name[10];  
    double grade;  
}; // blueprint (no memory yet)
```

2. Structure Declaration (creates a variable)

- A declaration creates a variable of that type and **allocates memory**.

```
struct student s1;
```

Total size: 24 bytes

s1	number (int) 4 bytes	20251234
	name (char[10]) 10 bytes	f i r s t _ l a s t \0
	grade (double) 8 bytes	4.3



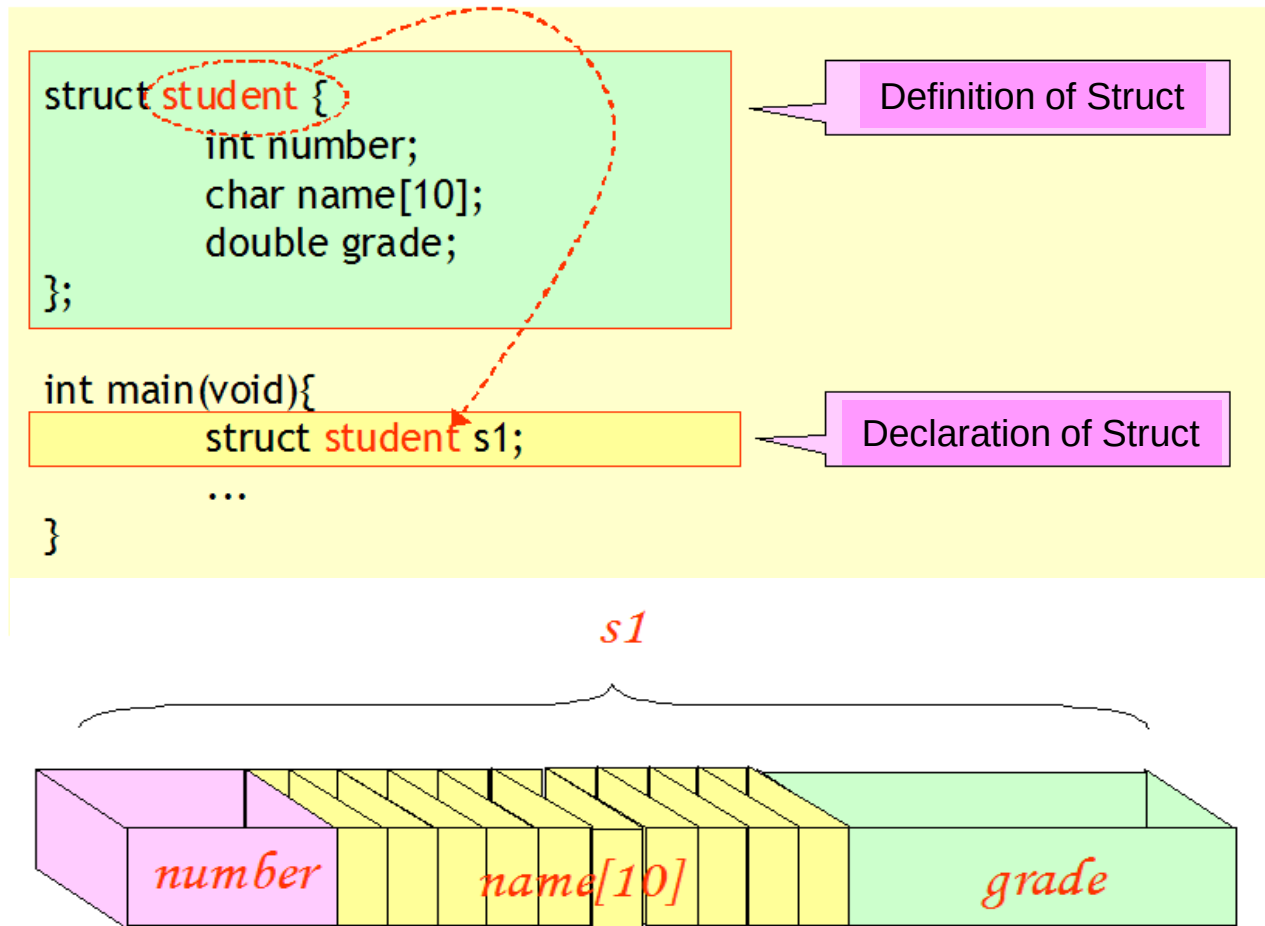
Like making an actual waffle from the mold.
A variable of the structure type is created
and **memory is allocated**.

Example

```
struct student s1;    // declaration + memory allocation  
s1.number = 20251234;  
strcpy(s1.name, "first last");  
s1.grade = 4.3;
```

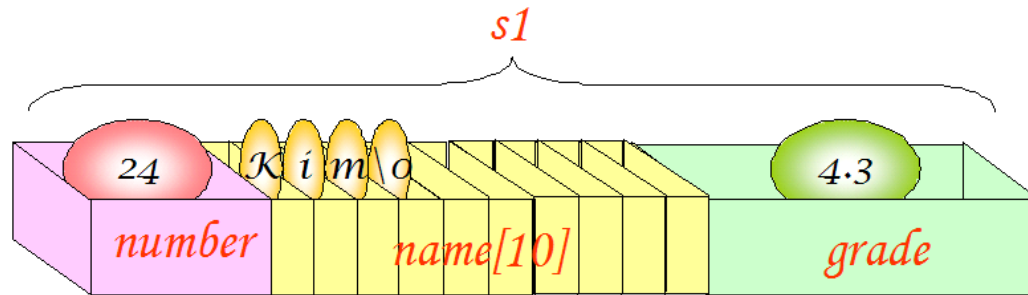

Structure Definition & Declaration

- Defining a structure and declaring a structure variable are different.



Structure Initialization

```
struct student {  
    int number;  
    char name[10];  
    double grade;  
};  
struct student s1 = { 24, "Kim", 4.3 }; // s1 is NOT a pointer,  
                                         // it's variable of derived data type
```



Example of a Structure Definition

```
// Screen coordinates
struct point {
    int x; // x coordinate
    int y; // y coordinate
};

// Complex number
struct complex {
    double real; // real part
    double imag ; // imaginary part
};

// date
struct date {
    int month;
    int day;
    int year;
};
```

Declaring and Initializing a Structure Variable

```
struct student {  
    int number;  
    char name[10];  
    double grade;  
};  
  
struct student s1 = { 24, "Kim" , 4.3 }; // s1 is NOT a  
                                          // pointer, it's a  
                                          // variable of derived type  
  
// . symbol is an operator used when referencing members  
printf("Number: %d\n", s1.number);
```


Structure Member Reference

Syntax

Accessing structure members

yes

structure variable

structure member

`s1.grade = 3.8;`

The dot (.) operator is used to access members of a structure.



. (dot) is an operator used when referencing members in a structure.

Example

```
struct student s1;  
s1.number = 20251234;  
strcpy(s1.name, "first last");  
s1.grade = 3.8;
```

• Example: structure layout

s1	number (int) 4 bytes	20251234	+0
	name (char[10]) 10 bytes	f i r s t _ l a s t \0	+4
	grade (double) 8 bytes	3.8	+14
Total size: 24 bytes			+24

Structure Member Reference Example

```
struct student {  
    int number;  
    char name[10];  
    double grade;  
};  
  
int main( void )  
{  
    struct student s;  
  
    printf ( "Student number: " );  
    scanf ( "%d" , & s.number );  
    printf ( "Name: " );  
    scanf ( "%s" , s.name);  
    printf ( "Grade: " );  
    scanf ( "%lf" , & s.grade );  
  
    printf ( "\nStudent number: %d\n" , s.number );  
    printf ( "Name: %s\n" , s.name);  
    printf ( "Grade: %.2f\n" , s.grade );  
    return 0;  
}
```

Four Ways to Initialize a Structure

```
struct point { int x; int y; };
```

```
struct point p = { 1, 2 };
```

```
struct point q = { .y = 2, .x = 1 };
```

```
struct point r = p;
```

```
r = ( struct point ) { 1, 2 };
```

```
printf( "p=(%d, %d)\n" , p.x, p.y);
```

```
printf( "q=(%d, %d)\n" , q.x, q.y);
```

```
// ① in member order
```

```
// ② by name - order doesn't matter!
```

```
// ③ structs CAN be copied with =
```

```
// ④ re-initialize after declaration
```

If you initialize only the first members, the rest become zero automatically.

A Structure That Has a Structure as a Member

- **Structure** can contain another structure as a member.

1. Define structure: **date**

```
struct date {  
    int year;  
    int month;  
    int day;  
};
```



2. Define structure: **student** (contains a structure)

```
struct student {  
    int number;  
    char name[10];  
    struct date d; // structure within structure  
    double grade;  
};
```

student contains:

number

name[10]

d (struct date)

grade

3. Declare a structure variable

```
struct student s1;
```

4. Access members (including nested members)

```
s1.d.year = 1983; // year of date  
s1.d.month = 03; // month of date  
s1.d.day = 29; // day of date
```

Memory layout of s1 (example)

Member		Size (bytes)		Offset
number (int)		4		0
name[10] (char[10])		10		4
d (struct date)	d.year (int)	4		14
	d.month (int)	4		18
	d.day (int)	4		22
grade (double)		8		26
Total size: 34 bytes				

A Structure That Has a Structure as a Member

```
struct date {  
    int year;  
    int month;  
    int day;  
};  
  
struct student {           // Structure declaration  
    int number;  
    char name[10];  
    struct date d ;        // structure within structure  
    double grade;  
};  
  
struct student s1;         // Declare structure variable  
s1.d.year = 1983;          // Member reference  
s1.d.month = 3;  
s1.d.day = 29;
```

Assignment and Comparison of Structure Variables

- Assignment of variables of the same **structure** IS possible, but **comparison is NOT possible**.
- **Struct** assignment is allowed because a **struct** is considered a single object
 - it copies the entire memory block byte by byte.

`p2 = p1;`

good way



Not pointer,
It's a variable of derived data type

`p2.x = p1.x;`
`p2.y = p1.y;`

You can do this



```
if( p1 == p2 )  
{  
    printf("p1 and p2 are equal.")  
}
```

compilation error



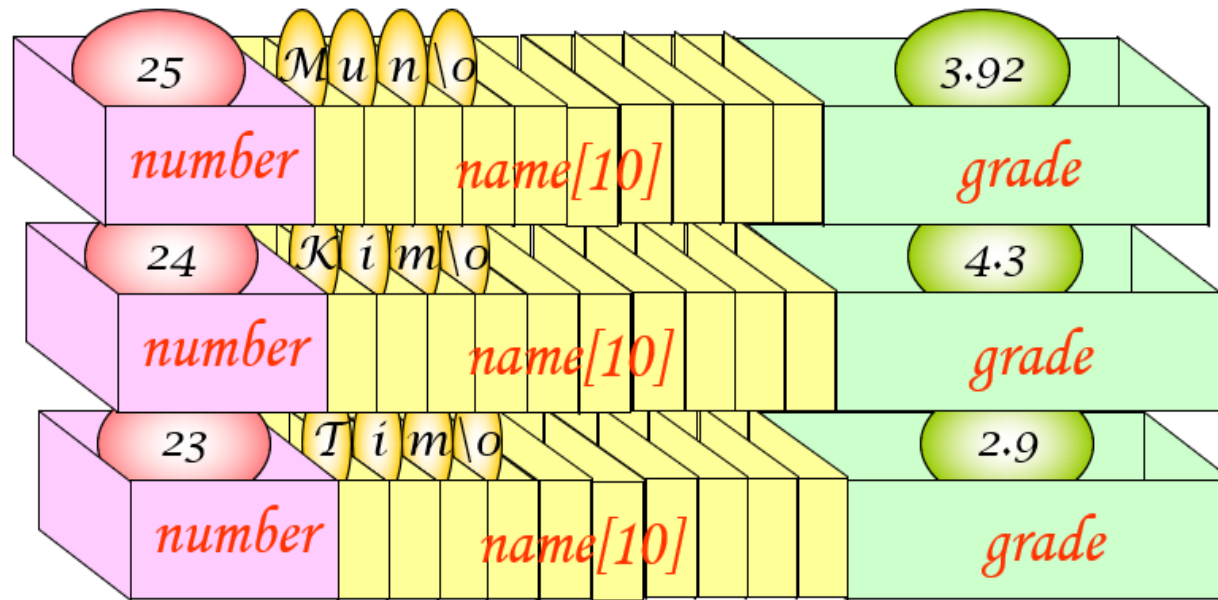
```
if( (p1.x == p2.x) && (p1.y == p2.y) )  
{  
    printf("p1 and p2 are equal.")  
}
```

right way



Array of Structures

- A collection of multiple structures.



Array of Structures

```
struct student {
    int number;
    char name[20];
    double grade;
};

int main( void )
{
    struct student list[100]; // array of structures

    list[2].number = 24;
    strcpy (list[2].name, "Hong Gil-dong" );
    list[2].grade = 4.3;
}

// Initialization:
struct student list[3] = {
    { 1, "Park", 3.42 }, { 2, "Kim", 4.31 }, { 3, "Lee", 2.98 }
};
// Counting elements: n = sizeof(list) / sizeof(list[0]);
```

Structures and Pointers

Structures and Pointers

1. Structure: student

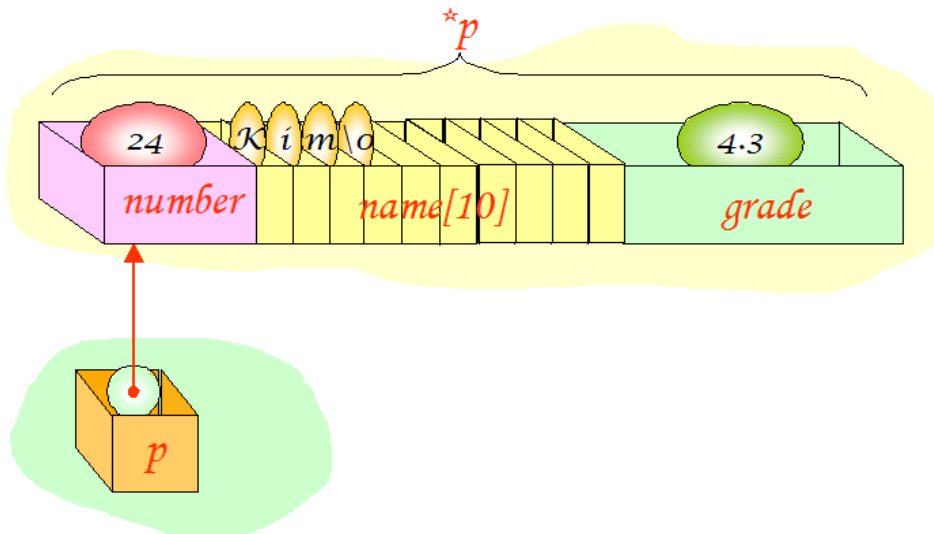
number	int
name	char name[10]
grade	double

A structure groups related variables of **different types**.

2. Declare structure variable and pointer

```
struct student *p;  
struct student s = { 24, "Kim", 4.3 };  
p = &s;  
printf("Student number =%d Name =%s Grade =%f \n", s.number, s.name, s.grade);  
printf("Student number =%d Name =%s Grade =%f \n", (*p).number, (*p).name, (*p).grade);
```

3. Memory representation



4. Key Note



- **p** holds the address of **s**.
- **(*p)** accesses the structure that **p** points to.

(*p).number is the same as **s.number**

(*p).name is the same as **s.name**

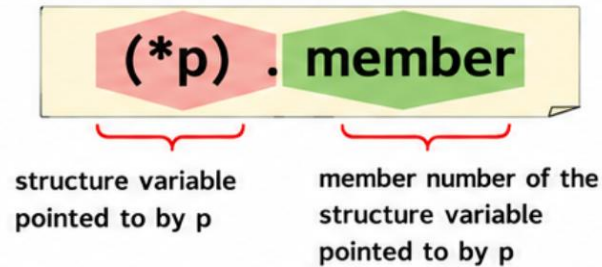
(*p).grade is the same as **s.grade**

Structures and Pointers: -> Operator

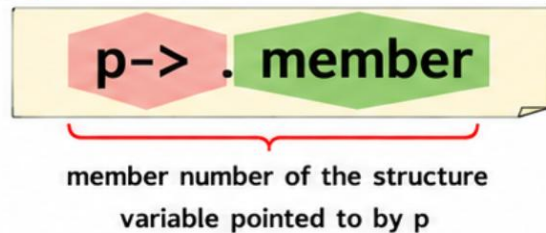
- -> operator is used when referencing a structure member with a structure pointer.

How -> works

Using **(*p).member**



Using **p->member**



```
struct student {  
    int number;  
    char name[10];  
    double grade;  
};  
  
struct student *p;  
struct student s = { 24, "Kim", 4.3 };  
  
p = &s;  
  
printf("Student number =%d Name =%s Grade =%f \n",  
       p->number, p->name, p->grade);
```

Key Points

- ✓ Use the **->** operator to access members of a structure through a pointer.
- ✓ **p** holds the address of a structure variable.
- ✓ **(*p).member** and **p->member** are equivalent.

Pointer to a Structure: -> Operator

```
struct student s = { 24, "Kim", 4.3 };
struct student *p;
p = &s;

// Two equivalent ways to access a member through a pointer:
printf("Number=%d Name=%s Grade=%f\n",
      s.number , s.name, s.grade );
printf("Number=%d Name=%s Grade=%f\n",
      (*p).number, (*p).name, (*p).grade);
printf("Number=%d Name=%s Grade=%f\n",
      p->number, p->name, p->grade);    // preferred!
```

-> is used when referencing a structure member with a structure POINTER; it means exactly the same as (*p).member.

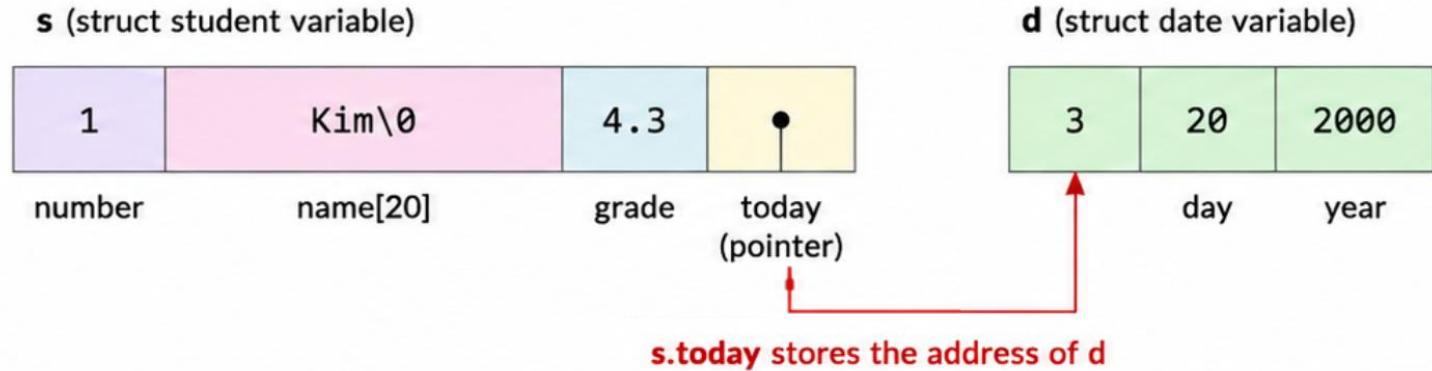
A Structure That Has a Pointer as a Member

```
struct date {
    int month;
    int day;
    int year;
};

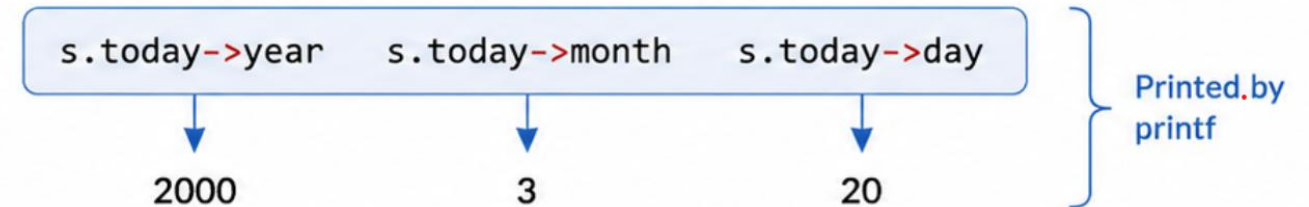
struct student {
    int number;
    char name[20];
    double grade;
    struct date *today; // pointer member
};

struct date d = { 3, 20, 2000 }; // a date variable
struct student s = { 1, "Kim", 4.3 }; // initialize student
s.today = &d; // store address of d in today
printf("Birth: Year %d month %d day %d\n",
       s.today->year, s.today->month, s.today->day);
```

1. Variables



2. Accessing through the pointer



Key Points

- ✓ A structure can have a pointer as one of its members.
- ✓ The pointer member stores the address of another variable (or structure).
- ✓ Use the **->** (arrow) operator to access members of the structure through the pointer.
- ✓ In this example, **s.today** points to **d**, so **s.today->year** returns 2000, **s.today->month** returns 3, and **s.today->day** returns 20.

A Structure That Has a Pointer as a Member

```
struct date {
    int month;
    int day;
    int year;
};

struct student {
    int number;
    char name[20];
    double grade;
    struct date *today ;
};

struct date d = { 3, 20, 2000 };
struct student s = { 1, "Kim" , 4.3 };
s.today = &d;
printf("Birth: Year %d month %d day %d\n",
       s.today->year, s.today->month, s.today->day);
```

Structures and Functions: Parameters

1. Passing a structure (by value)

- A **copy** of the structure is passed.
- More time and memory if the structure is large.

```
int equal(struct student s1, struct student s2)
{
    if (s1.number == s2.number)
        return 1;
    else
        return 0;
}

int main(void)
{
    struct student a = {1,"lee",3.8};
    struct student b = {2,"kim",4.0};
    if (equal(a,b) == 1)
        printf("Same student \n");
    else
        printf("Other students \n");
}
```



Copies of **a** and **b** are passed to the function.

2. Passing a pointer to a structure (by reference)

- The **address** of the structure is passed.
- **Saves** time and space, but the original may be modified.

```
int equal(struct student *p1, struct student *p2)
{
    if (p1->number == p2->number)
        return 1;
    else
        return 0;
}

int main(void)
{
    struct student a = {1,"lee",3.8};
    struct student b = {2,"kim",4.0};
    if (equal(&a,&b) == 1)
        printf("Same student \n");
    else
        printf("Other students \n");
}
```

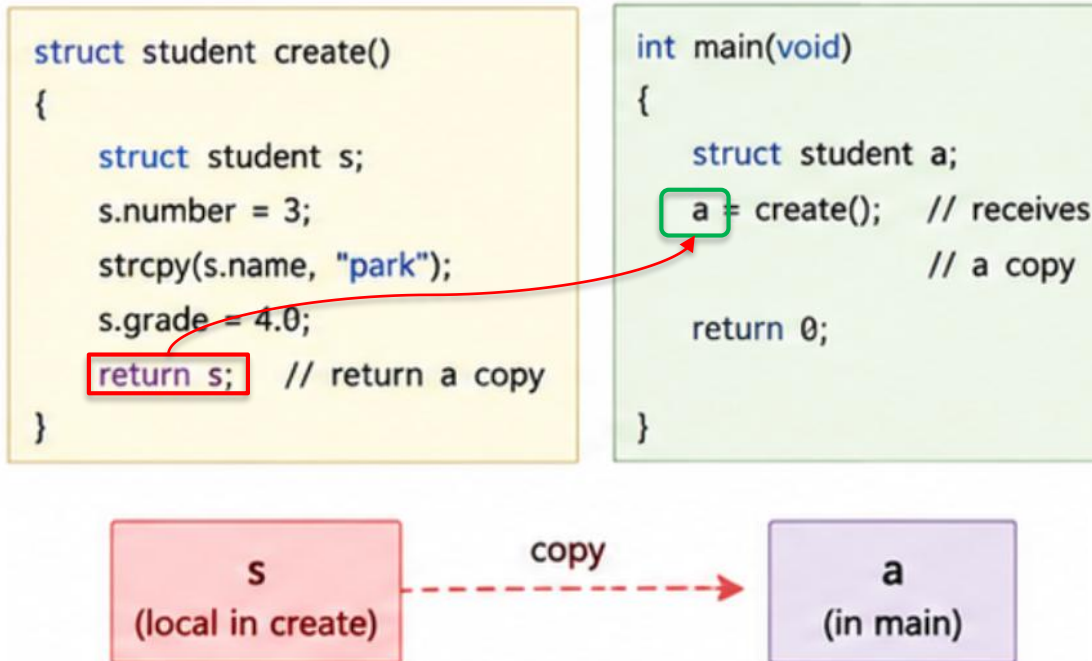


Addresses of **a** and **b** are passed to the function.

Structures and Functions: Return Value

● 3. Returning a structure (by value)

- A **copy** of the structure is returned to the caller.



Passing a structure (by value):

A copy of the entire structure is passed to the function.



Passing a pointer to a structure (by reference):

The address of the structure is passed, so the function accesses the original data.



Returning a structure:

A copy of the structure is returned from the function to the caller.

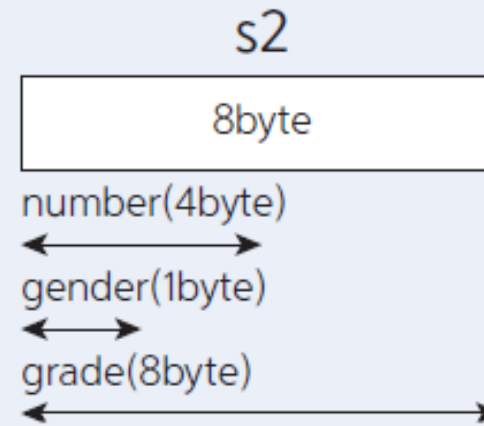
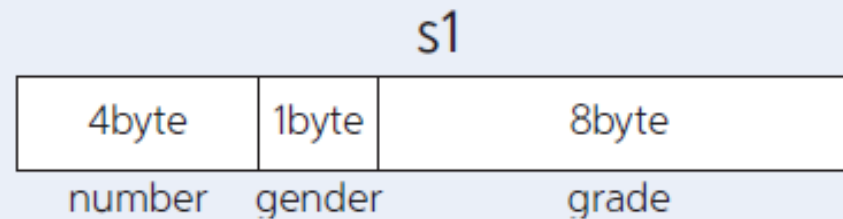
User-Defined Types in C

union

- **union**: declared and used very similarly to a **structure**, but ALL members share the same memory (only one is valid at a time).

```
struct student
{
    int number;
    char gender;
    double grade;
} s1;
```

```
union student
{
    int number;
    char gender;
    double grade;
} s2;
```



enum

Syntax

```
enum EnumName {  
    CONST1, CONST2, CONST3, ...  
    /* optional: = value */  
};
```

enum keyword name of the enumeration enumeration constants (symbolic names)

An **enumeration (enum)** is a user-defined data type that consists of a set of **named integer constants**.

- ✓ Each name (constant) has an integer value.
- ✓ The first value is 0 by default, and subsequent values increase by 1.
- ✓ You can assign a specific value to any constant. Example: SUN = 1, MON = 5, ...
- ✓ **enum** variables can store only the values defined in the **enum**.

Example

```
enum days { SUN, MON, TUE, WED,  
            THU, FRI, SAT };
```

```
enum days today;  
today = SUN; // OK!
```

today is a variable
of type **enum** days
and can store any
of the constants.

Underlying values (default)

SUN	MON	TUE	WED	THU	FRI	SAT
0	1	2	3	4	5	6

Compiler assigns integer values starting from 0 and increasing by 1.

The constant SUN has the value 0. So when today = SUN, the value 0 is stored in memory.

Key Points



enum is a user-defined type made of named integer constants.



By default, the first constant has value 0, and the next ones increase by 1.



You can assign a specific value to any constant.



enum variables can store only the values defined in the **enum**.

Why Use enum?

1. Use of integers

Use plain integer values.

```
switch (code) {  
    case 1:  
        printf("LCD TV\n");  
        break;  
    case 2:  
        printf("OLED TV\n");  
        break;  
}
```



Computers are easy to understand, but people have difficulty remembering.

2. Symbolic Constant (#define)

Use symbolic names with `#define`.

```
#define LCD 1  
#define OLED 2  
  
switch (code) {  
    case LCD:  
        printf("LCD TV\n");  
        break;  
    case OLED:  
        printf("OLED TV\n");  
        break;  
}
```



When writing symbolic constants, easy to make duplication errors.

3. Enumeration (enum)

Use `enum` to define a new data type with named integer constants.

```
enum tvtype { LCD, OLED };  
  
enum tvtype code;  
code = LCD;  
  
switch (code) {  
    case LCD:  
        printf("LCD TV\n");  
        break;  
    case OLED:  
        printf("OLED TV\n");  
        break;  
}
```



The compiler checks to ensure that no duplication occurs.

Key Point

- ✓ Integers are simple but hard to remember and less readable.
- ✓ `#define` gives names to values, improving readability, but the compiler does not check for duplicates.
- ✓ `enum` creates a new type with named constants, improves type safety, and the compiler prevents duplicates.

typedef

typedef

- **typedef** creates a **new name (an alias)** for an existing type.
- It does *not* create a new type—just a new name for convenience.

Syntax

```
typedef existing_type new_name;
```

Existing data type

New name (alias)

Examples

Example 1: Simple type

```
typedef unsigned char BYTE;
```



Now **BYTE** can be used instead of **unsigned char**.

```
BYTE data; // same as: unsigned char data;
```

Example 2: Structure type

```
typedef struct student {  
    int number;  
    char name[20];  
    double grade;  
} Student;
```



Now **Student** can be used instead of **struct student**.

```
Student s; // same as: struct student s;
```

Key Points



typedef *does not* create a new type; it only creates a new name for an existing type.



It improves code readability and makes declarations easier to write.



Commonly used with complex types such as structures, pointers, and function pointers.

typedef with Structure

```
// Without typedef:
struct Person {
    char name[50];
    int age;
};
struct Person s1 = {"Nora", 27};

// With typedef (recommended style):
typedef struct {
    char name[50];
    int age;
} Person;           // Person is now a type name

Person s1 = {"Nora", 27}; // no need to write 'struct'!
```

Common Mistakes

- Use `.` for a struct variable and `->` for a pointer to a struct
 - `p->member` is shorthand for `(*p).member`.
- Whole-structure assignment (`=`) copies every member, but structs cannot be compared directly with `==`.
- Structures are often passed to functions **by pointer** to avoid copying the whole structure.
- **`typedef struct {...} Name;`** lets you drop the struct keyword afterward
 - a common convenience in real code.

Q & A

Lab & Quiz: right after this - see you in the practice session!